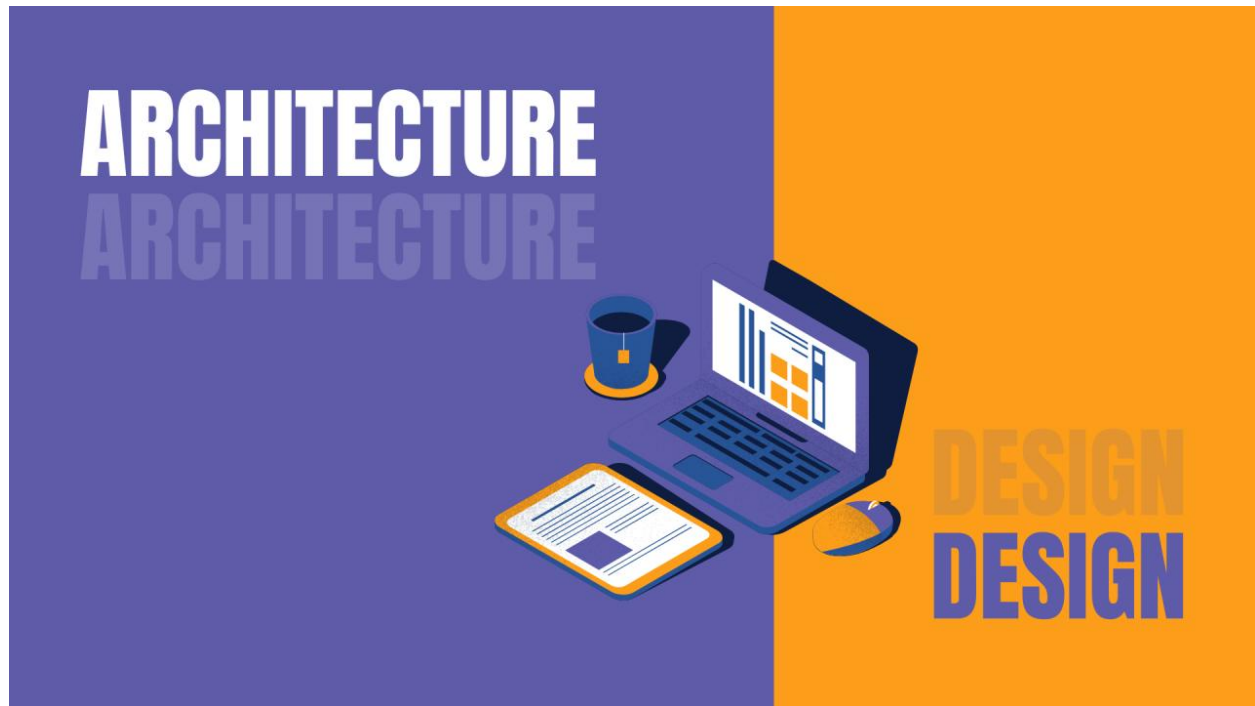


Architectuurdocument



Studenten: Nawiesh Ragho SE/1123/064
Aditije Matai SE/1123/043
Jozua Mertowirijo SE/1123/045
Reshwan Moenna SE/1123/047
Misael Karwofodi SE/1123/033

Opleiding: Software Engineering

Programmaonderdelen: Ontwikkel en deploy een productie klare chatbot

Docenten: Rwynn Christian

Debora Bergwijn

Datum: 12 Maart 2026

Inhoudsopgave

Inleiding.....	3
1. Systeemoverzicht.....	4
1 Architectuurdiagram.....	4
1.1 Uitleg van de architectuur	5
2. Gebruikte Componenten	6
2.1 Frontend.....	6
2.2 Backend.....	6
2.3 AI Database	6
2.4 AI Model.....	7
3. Onderbouwing van keuzes.....	7
3.1 API-gebaseerde AI.....	7
3.2 Knowledge Base	7
3.3 Geen volledige RAG architectuur.....	8
4. Datastromen	8
4.1 Login.....	8
4.2 Chatverzoek	8
4.3 Data ophalen.....	9
5. Afhankelijkheden en Schaalbaarheid.....	10
5.1 Afhankelijkheden	10
5.2 Schaalbaarheid.....	10
6. Kosten, Risico's en Beperkingen	10
6.1 Kosten	10
6.2 Risico's.....	10
6.3 Beperkingen	11

Inleiding

Dit document beschrijft de architectuur van het systeem UNASAT AI Student Assistant. Het systeem is ontwikkeld als onderdeel van een schoolproject waarbij de focus ligt op het integreren van een AI-chatbot in een webapplicatie.

Het doel van deze applicatie is om studenten te ondersteunen met een digitale assistent die vragen kan beantwoorden en aanvullende informatie kan tonen zoals cijfers en roostergegevens.

In dit document wordt uitgelegd hoe het systeem technisch is opgebouwd. Daarbij wordt gekeken naar de architectuur van het systeem, de gebruikte componenten, de datastromen en de technische keuzes die zijn gemaakt tijdens de ontwikkeling.

Dit document is geschreven zodat een derde partij het systeem kan begrijpen zonder verdere uitleg van de ontwikkelaars.

1. Systeemoverzicht

De **UNASAT AI Student Assistant** is een webapplicatie waarin studenten via een dashboard toegang hebben tot een AI-chatbot en andere studiegerelateerde informatie.

De belangrijkste functies van het systeem zijn:

- Studenten kunnen inloggen in een webportaal
- Studenten kunnen vragen stellen aan een AI-chatbot
- Studenten kunnen hun cijfers bekijken
- Studenten kunnen hun rooster bekijken
- Chatgesprekken worden opgeslagen
- Studenten kunnen feedback geven op AI-antwoorden
- Een administrator kan gebruikers beheren

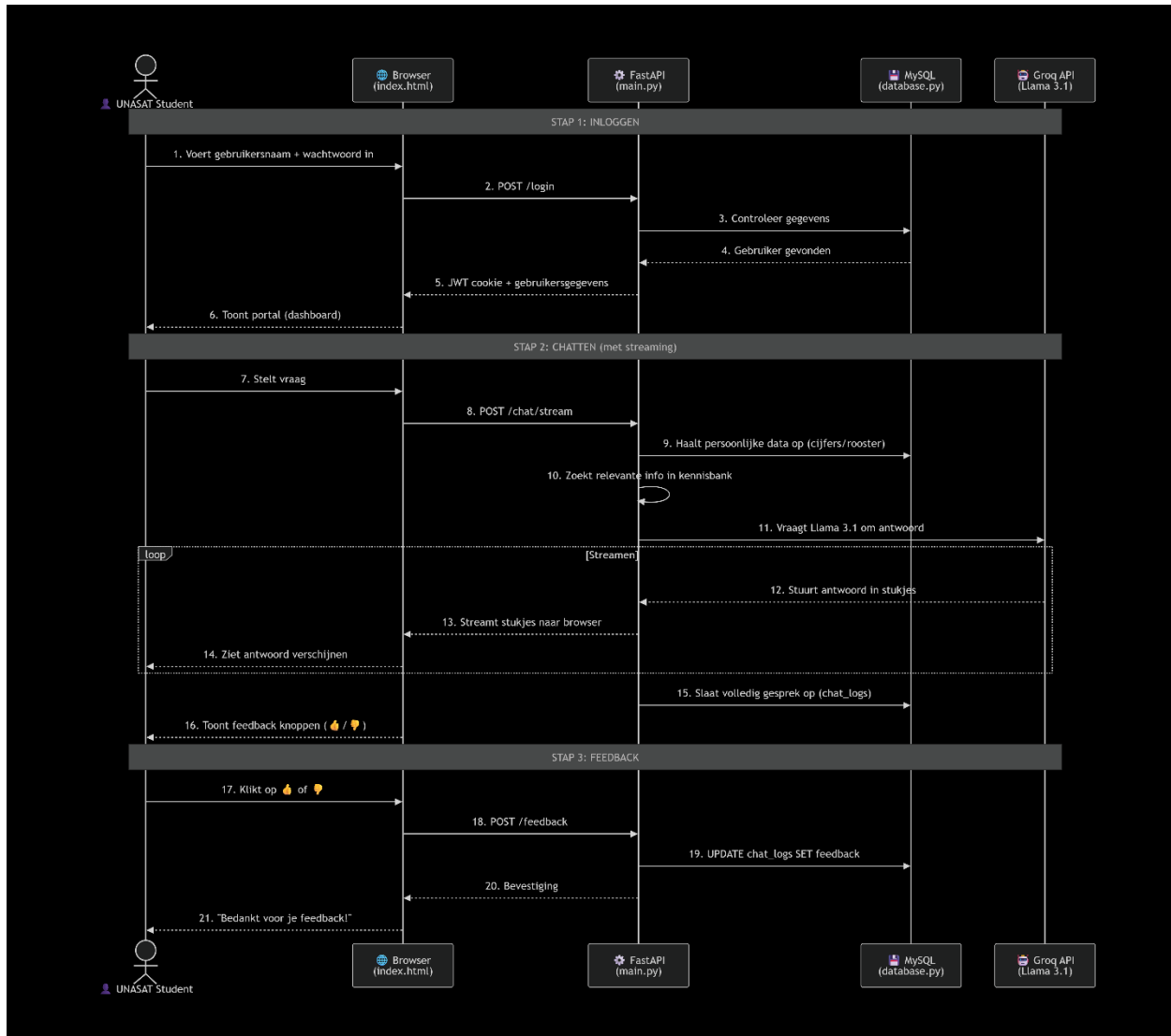
Het systeem is opgebouwd uit verschillende onderdelen:

- een **frontend** (webinterface)
- een **backend API**
- een **database**
- een **AI-model via een API**

Deze onderdelen werken samen om de functionaliteit van het systeem mogelijk te maken.

1 Architectuurdiagram

De architectuur van het systeem bestaat uit vier hoofdcomponenten.



1.1 Uitleg van de architectuur

1. De student gebruikt een browser om toegang te krijgen tot het dashboard.
2. De frontend stuurt verzoeken naar de backend via HTTP API-calls.
3. De backend verwerkt de logica van het systeem.
4. De backend haalt gegevens op uit de database.
5. Voor AI-vragen wordt een request gestuurd naar een AI-model via een API.
6. Het antwoord wordt teruggestuurd naar de gebruiker.

Deze architectuur maakt het systeem overzichtelijk en modulair.

2. Gebruikte Componenten

2.1 Frontend

De frontend van het systeem is ontwikkeld met:

- HTML
- CSS
- JavaScript

De frontend draait in de browser en bevat:

- loginpagina
- student dashboard
- chatbot interface
- pagina's voor cijfers en rooster
- admin interface

De frontend communiceert met de backend via HTTP requests.

2.2 Backend

De backend van het systeem is ontwikkeld met **FastAPI in Python**.

De backend zorgt voor:

- authenticatie
- verwerking van chatverzoeken
- ophalen van cijfers
- ophalen van roosterinformatie
- opslaan van chatlogs
- verwerken van feedback
- beheer van gebruikers

FastAPI is gekozen omdat het snel te ontwikkelen is en goed geschikt is voor API-gebaseerde systemen.

2.3 AI Database

Voor de opslag van gegevens wordt **MySQL** gebruikt.

De database bevat tabellen voor:



- gebruikers
- cijfers
- roosterinformatie
- chatlogs
- feedback

2.4 AI Model

Voor het genereren van antwoorden wordt een **extern AI-model via een API** gebruikt.

Het model ontvangt:

- de vraag van de student
- context uit de knowledge base
- systeeminstructies

Het model genereert vervolgens een antwoord dat wordt teruggestuurd naar de gebruiker.

3. Onderbouwing van keuzes

Tijdens het ontwerp van het systeem zijn verschillende technische keuzes gemaakt.

3.1 API-gebaseerde AI

Er is gekozen voor een AI-model via een API in plaats van een lokaal getraind model.

Voordelen hiervan zijn:

- snellere implementatie
- minder infrastructuur nodig
- eenvoudiger onderhoud
- geschikt voor een prototype

3.2 Knowledge Base

Het systeem gebruikt een tekstbestand met informatie over UNASAT als **knowledge base**.

Deze knowledge base wordt toegevoegd aan de prompt van het AI-model zodat het model antwoorden kan genereren die beter aansluiten op de context van het systeem.

3.3 Geen volledige RAG architectuur

Een volledige **Retrieval-Augmented Generation (RAG)** architectuur met embeddings en een vector database is niet gebruikt.

De reden hiervoor is dat dit te complex zou zijn voor de scope van het project. In plaats daarvan is gekozen voor een eenvoudiger systeem met een knowledge base tekstbestand.

4. Datastromen

4.1 Login

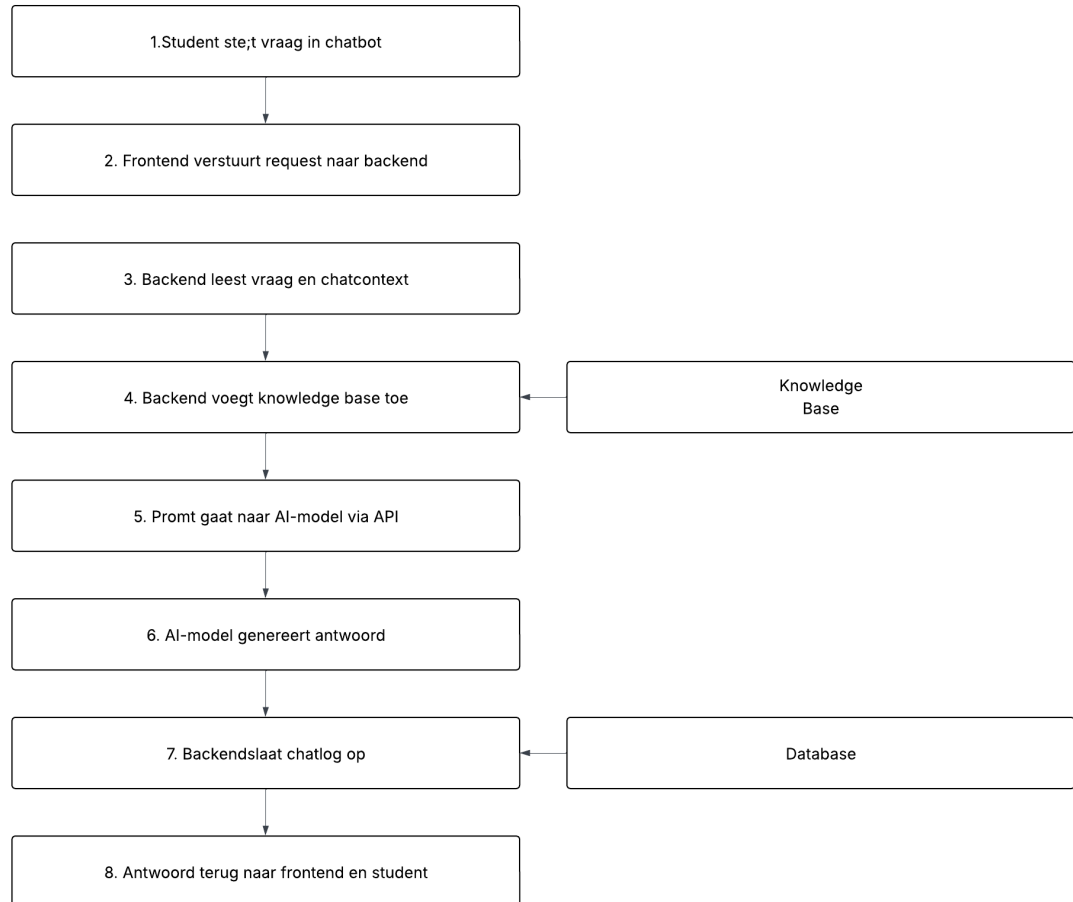
1. De gebruiker voert zijn gebruikersnaam en wachtwoord in.
2. De frontend stuurt deze gegevens naar de backend.
3. De backend controleert de gegevens in de database.
4. Bij een succesvolle login krijgt de gebruiker toegang tot het dashboard.

4.2 Chatverzoek

1. De student stelt een vraag in de chatbot.
2. De vraag wordt naar de backend gestuurd.
3. De backend voegt context toe uit de knowledge base.
4. De backend stuurt een prompt naar het AI-model.
5. Het AI-model genereert een antwoord.
6. Het antwoord wordt teruggestuurd naar de frontend.

Datastroombiagram - AI Chat Flow

Stap-voor-stap weergave van hoe een chatvraag door het systeem stroom.



4.3 Data ophalen

Voor functies zoals cijfers en rooster:

1. De frontend stuurt een request naar de backend.
2. De backend haalt de gegevens op uit de database.
3. De gegevens worden teruggestuurd naar de frontend.

5. Afhankelijkheden en Schaalbaarheid

5.1 Afhankelijkheden

Het systeem is afhankelijk van verschillende technologieën:

- Python
- FastAPI
- MySQL
- Docker
- JavaScript
- AI API

Daarnaast is een internetverbinding nodig voor het communiceren met het AI-model.

5.2 Schaalbaarheid

De huidige architectuur maakt het mogelijk om het systeem later uit te breiden.

Mogelijke verbeteringen zijn:

- gebruik van een vector database
- implementatie van een volledige RAG architectuur
- caching van veelgestelde vragen
- load balancing van de backend
- verbeterde beveiliging en authenticatie

6. Kosten, Risico's en Beperkingen

6.1 Kosten

De belangrijkste kosten van het systeem zijn:

- hosting van de backend
- database hosting
- AI API gebruik
- Een eenvoudige implementatie zou ongeveer tussen **15 en 60 USD per maand** kunnen kosten afhankelijk van het gebruik

6.2 Risico's

Belangrijke risico's zijn:



- afhankelijkheid van een externe AI API
- mogelijke onnauwkeurigheid van AI-antwoorden
- beperkte beveiliging in de prototypeversie
- afhankelijkheid van een internetverbinding

6.3 Beperkingen

Omdat dit systeem een prototype is, zijn er enkele beperkingen:

- geen volledige RAG architectuur
- beperkte beveiliging
- eenvoudige frontend
- beperkte schaalbaarheid

Deze punten kunnen in een latere versie van het systeem verder verbeterd worden.